ENGRAM

Permanent decentralized memory for AI agents

- Bittensor testnet subnet 450
- Content-addressed semantic memory for text, images, PDFs, and URLs
- Miners store and serve memories
- Validators score recall, latency, and proof success
- Arweave archives raw media for long-term permanence
- Demo: https://theengram.space
- Repo: https://github.com/Dipraise1/Engram

One-Sentence Concept

What Engram is

- Engram is a decentralized AI memory layer built as a Bittensor subnet.
- It replaces fragile centralized vector databases with a network of miners that store, search, and prove AI memories.
- Each memory gets a deterministic CID, so it can be referenced by what it encodes, not by where it is hosted.
- The goal is portable, verifiable, permanent memory for AI agents and retrieval systems.

The Problem

AI memory is becoming critical infrastructure

- AI agents, copilots, and RAG systems depend on vector memory to remember documents, conversations, tools, and user context.
- Most memory today sits in centralized vector databases or local indexes.
- If the provider disappears, rate-limits, changes pricing, loses data, or suffers an outage, the agent loses its memory.
- Users usually cannot verify that stored vectors still exist or that retrieved results match what was stored.
- This creates lock-in, data fragility, and trust assumptions at the exact layer agents depend on most.

Why Now

Agents make memory a first-order problem

- AI systems are moving from single prompts to long-running agents.
- Long-running agents need persistent memory across sessions, models, apps, and infrastructure providers.
- Semantic memory is hard to migrate because providers use different index formats, embeddings, metadata schemas, and retrieval APIs.
- Permanent storage networks have solved durable bytes, but not incentivized semantic search over embeddings.
- Bittensor makes it possible to turn useful AI infrastructure into an open incentive market.

Engram's Solution

Content-addressed memory with economic incentives

- Convert content into embeddings and metadata.
- Generate a deterministic Engram CID from the embedding and metadata.
- Store the embedding across miners instead of one central database.
- Query the network semantically and retrieve ranked results by CID.
- Challenge miners to prove they still hold stored vectors.
- Reward miners and validators through Bittensor based on measurable utility.

How The Subnet Works

The incentive loop

- Users and apps ingest memory or query memory.
- Miners provide storage, indexing, retrieval, and proof responses.
- Validators test miners for semantic recall, query latency, and proof success.
- Bittensor weights direct emissions toward better-performing miners.
- Miners compete on reliability and usefulness instead of promises.
- The protocol becomes a marketplace for persistent AI memory.

Product Surface

Built for developers and operators

- Python SDK for ingest, query, media upload, namespaces, and integrations.
- CLI for storing text, batch ingestion, querying, and network status.
- Next.js playground for text, image, PDF, and URL ingestion.
- CID explorer for looking up stored memories and media proof metadata.
- Miner and validator entry points for subnet participants.
- Rust core for CID and proof primitives with Python fallback paths.

Technical Architecture

Components already in the repo

- Miner: embeds content, stores vectors, serves queries, answers proof challenges.
- Validator: samples ground truth, queries miners, challenges storage, sets weights.
- engram-core: Rust/PyO3 core for deterministic CIDs and storage proof primitives.
- Vector store: persistent nearest-neighbor index for semantic search.
- Storage layer: replication routing and Arweave-backed media archival.
- Web app: playground, dashboard, CID pages, and public demo surface.
- SDK: Python client plus LangChain and LlamaIndex integration paths.

Trust Model

What users should be able to verify

- CID determinism: the same embedding and metadata produce the same memory identifier.
- Storage proof: miners can be challenged to prove they hold the vector behind a CID.
- Validator scoring: miners are rewarded for recall, latency, and proof success.
- Namespace auth: private memory access can be tied to signed wallet ownership.
- Media permanence: raw files can be archived outside the miner on Arweave.
- Roadmap: harden proof keying, threshold private namespaces, replication repair, and production request signing.

Why Bittensor

Engram is a protocol, not only an app

- A normal app can host a vector database.
- A subnet can create a competitive market for many independent storage and retrieval providers.
- Miners earn only if validators can measure useful service.
- Validators can compare miners continuously instead of trusting static claims.
- Stakers and ecosystem participants can support the subnet if they believe the memory service is valuable.
- This aligns Engram with open AI infrastructure rather than a single hosted SaaS provider.

Why Arweave

Semantic memory plus permanent bytes

- Engram CIDs address semantic memory.
- Arweave transaction IDs address raw files such as images, PDFs, and archived pages.
- Together, Engram can search meaning while Arweave preserves original media.
- Private namespaces can encrypt raw bytes before upload.
- This creates a two-layer memory model: searchable vectors on miners, durable media on permanent storage.

Use Cases

Who needs Engram

- AI agents that need memory across sessions and infrastructure providers.
- RAG teams that want portable retrieval records and verifiable storage.
- Research archives that need searchable, durable public knowledge.
- DAOs and crypto apps that need decentralized AI context storage.
- Data communities that want public datasets stored once and queried many times.
- Developers who want a memory layer that is not tied to one vector database vendor.

Competitive Positioning

Engram's lane

- Pinecone, Weaviate, Qdrant, and Chroma are strong vector database tools.
- Arweave, Filecoin, and IPFS focus on durable content storage and retrieval.
- Engram combines semantic memory, Bittensor incentives, miner competition, validation, and storage proofs.
- It should not compete by being the easiest hosted vector DB.
- It should compete by being decentralized, verifiable, portable, and aligned with agent memory.

Current Status

What is real today

- Testnet subnet UID 450.
- Working miner, validator, SDK, CLI, Rust core, and web app.
- Content-addressed CIDs for embeddings.
- HMAC challenge-response storage proofs.
- Arweave media archival path for images, PDFs, and URLs.
- Namespace authentication and private-memory work in progress.
- Public dashboard and playground at theengram.space.
- Active hardening issues for security, replication, packaging, and web request signing.

Roadmap

The next build sequence

- 0 to 30 days: merge critical fixes, keep CI green, add license/package cleanup, improve deployment docs, publish mainnet-readiness checklist.
- 30 to 90 days: harden storage proofs, implement production web request signing, improve miner monitoring, add replication repair worker.
- 90 to 180 days: threshold private namespaces, broader SDK examples, mainnet candidate testing, partner pilots, public dataset demos.
- Success metric: a subnet that validators can measure, miners can run, developers can integrate, and funders can understand.

Funding Ask

Milestone-based support

- Mainnet readiness sprint: CI, packaging, deployment, docs, and operational hardening.
- Proof hardening sprint: stronger proof design, Rust/PyO3 parity, validator compatibility, security notes.
- Private memory sprint: threshold decryption design, namespace tests, SDK docs, updated threat model.
- Replication reliability sprint: repair worker, degraded CID recovery, miner churn simulation, dashboard metrics.
- Arweave permanence sprint: encrypted media defaults, retrieval UX, examples, and proof visibility.
- Contributor bounty pool: small scoped tasks for docs, tests, SDK integrations, and miner onboarding.

Collaboration Ask

The people Engram needs next

- Bittensor operators to run miners and validators and advise on subnet economics.
- Rust and cryptography engineers to harden proof primitives and PyO3 boundaries.
- Distributed systems engineers to improve replication, health checks, and failure recovery.
- Security reviewers to examine request signing, namespace auth, and private memory.
- AI integrations engineers to build agent, LangChain, LlamaIndex, and RAG examples.
- Developer relations collaborators to turn the project into clear guides, posts, demos, and grant materials.

Opportunity Map

Where support can come from

- Bittensor ecosystem: validators, stakers, subnet operators, miners, and accelerators.
- Yuma: Bittensor-focused acceleration, technical help, compute, go-to-market, and capital network.
- Arweave ecosystem: permanent media storage, encrypted archival, and proof/retrieval UX.
- ao Ventures: Arweave/ao-aligned builder support and investor exposure.
- Gitcoin/public goods: open-source AI memory and decentralized infrastructure.
- Filecoin grants: relevant if Engram adds Filecoin storage or retrieval integration.
- AI agent teams: paid pilots for persistent memory, private namespaces, and SDK integrations.

Why Support Engram

The thesis

- AI memory will become infrastructure.
- Centralized memory creates lock-in and fragility.
- Decentralized storage alone does not solve semantic retrieval.
- Bittensor can incentivize useful AI memory providers.
- Engram already has a live testnet implementation.
- Funding now should turn a working subnet into a credible mainnet-ready protocol.

Next Steps

Concrete asks

- Sponsor one milestone instead of the whole roadmap.
- Run a miner or validator on testnet and report operational issues.
- Review the protocol, storage proof model, and namespace security.
- Integrate Engram into one real agent or RAG workflow.
- Fund a public dataset demo that stores media on Arweave and semantic memory on Engram.
- Use the live demo and repo as the diligence starting point.

References

Links for diligence

- Website: https://theengram.space
- GitHub: https://github.com/Dipraise1/Engram
- Bittensor subnet docs: https://docs.learnbittensor.org/subnets/understanding-subnets
- Bittensor emissions docs: https://docs.learnbittensor.org/learn/emissions
- Yuma services: https://www.yumaai.com/services
- Arweave funding: https://www.arweave.org/funding
- ao Ventures: https://www.aoventures.io
- Gitcoin grants: https://grants.gitcoin.co
- Filecoin ProPGF: https://filpgf.io/propgf